

Sistema de tipos - MyPy

TUIA - Programación 2

2025 - 2C

- Qué es MyPy?
- Tipado dinámico
- Tipado estático
- Instalación
- Tipos
- Modo `strict`

- Python es un lenguaje dinámicamente tipado.

- Python es un lenguaje dinámicamente tipado.
- Mypy es un verificador de tipos estáticos para Python.

- Python es un lenguaje dinámicamente tipado.
- Mypy es un verificador de tipos estáticos para Python.
- Actúa como un **linter**: *una herramienta de análisis de código estático para detectar posibles errores de programación, fallos, errores de estilo, etc.*

- Python es un lenguaje dinámicamente tipado.
- Mypy es un verificador de tipos estáticos para Python.
- Actúa como un **linter**: *una herramienta de análisis de código estático para detectar posibles errores de programación, fallos, errores de estilo, etc.*
- Todo lo que hace MyPy es verificar las sugerencias de tipo (*type annotations*).

- Python es un lenguaje dinámicamente tipado.
- Mypy es un verificador de tipos estáticos para Python.
- Actúa como un **linter**: *una herramienta de análisis de código estático para detectar posibles errores de programación, fallos, errores de estilo, etc.*
- Todo lo que hace MyPy es verificar las sugerencias de tipo (*type annotations*).
- No compila el código fuente.

example.py

```
1 def resta(a, b):  
2     return a - b  
3  
4 resta('hola', 'chau')
```

output

```
$ python example.py  
Traceback (most recent call last):  
[...]  
    return a - b  
           ~ ~ ^ ~ ~  
TypeError: unsupported operand type(s) for -:  
    'str' and 'str'
```

example.py

```
1 def resta(a: int, b: int) -> int:
2     return a - b
3
4 resta('hola', 'chau')
```

output

```
$ mypy example.py
example.py:4: error: Argument 1 to "resta" has
    incompatible type "str"; expected "int" [arg-type]
example.py:4: error: Argument 2 to "resta" has
    incompatible type "str"; expected "int" [arg-type]
Found 2 errors in 1 file (checked 1 source file)
```

```
$ sudo apt-get install mypy  
[...]
```

```
$ mypy --version  
mypy 1.9.0 (compiled: yes)
```

- Instalar extensión: Mypy Type Checker

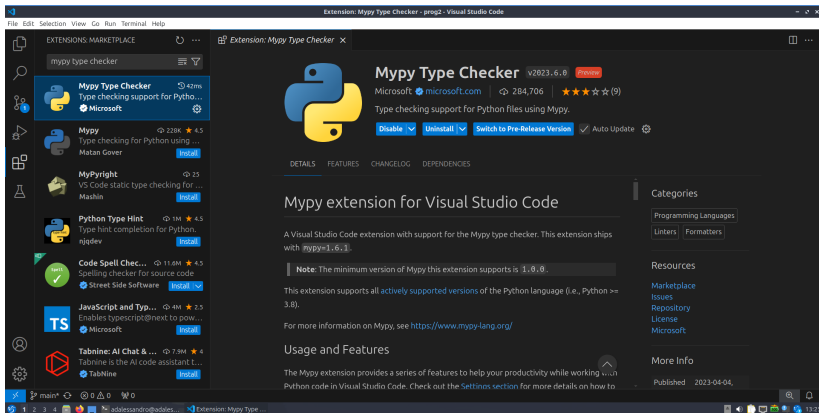


Figure 1: Mypy Type Checker

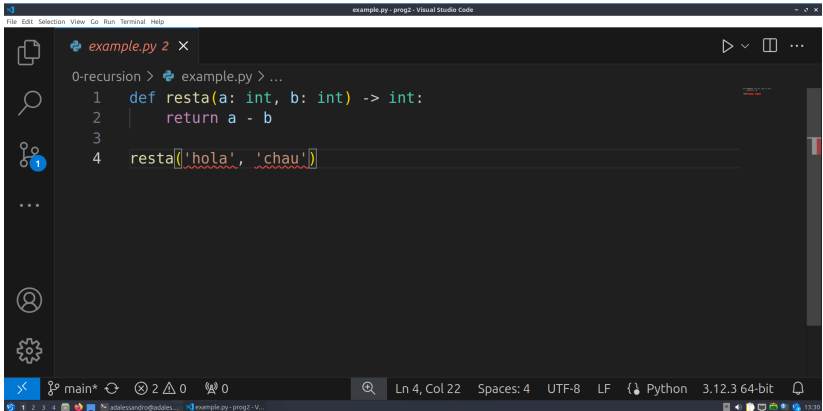


Figure 2: VSCode MyPy Error

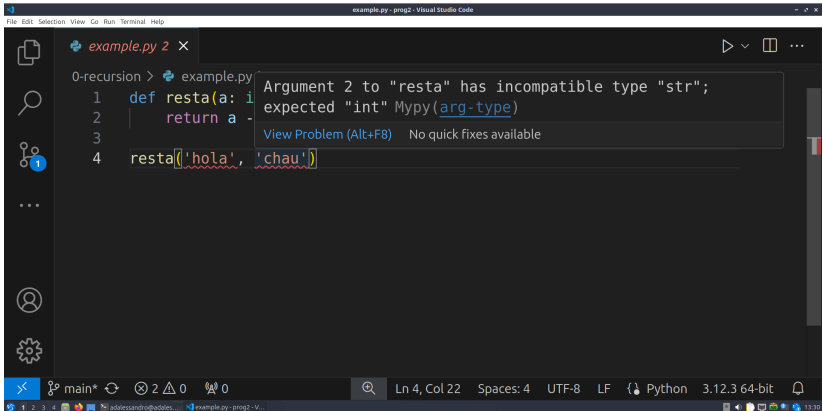


Figure 3: VSCode Mypy Error

- **int**: Número entero de tamaño arbitrario.

- **int**: Número entero de tamaño arbitrario.
- **float**: Número flotante.

- **int**: Número entero de tamaño arbitrario.
- **float**: Número flotante.
- **bool**: Valor booleano (True o False).

- **int**: Número entero de tamaño arbitrario.
- **float**: Número flotante.
- **bool**: Valor booleano (True o False).
- **str**: String Unicode.

- **int**: Número entero de tamaño arbitrario.
- **float**: Número flotante.
- **bool**: Valor booleano (True o False).
- **str**: String Unicode.
- Estos ya los conocemos, pero hay más. . .

- **Any**: Cualquier valor (tipado dinámico).
- **None**: Solo el valor `None`.
- **list[T]**: Lista de objetos del tipo `T`.
- **dict[K, V]**: Diccionario de objetos de tipo `K` hacia objetos de tipo `V`.
- **tuple[T1, ..., Tn]**: *N-tupla* con elementos de los tipos `T1, ..., Tn`.

El tipo `Any` y los constructores `List` y `Dict` están definidos en el módulo `typing` que viene junto con `MyPy`.

Más info: https://mypy.readthedocs.io/en/stable/kinds_of_types.html

- **Any**: Cualquier valor (tipado dinámico).

example.py

```
1 from typing import Any
2
3 def imprimir(x: Any):
4     print(x)
5
6 imprimir(1)
7 imprimir('a')
8 imprimir(True)
9 imprimir([1, 2, 3])
```

mypy

```
$ mypy example.py
Success: no issues found in 1 source file
```

- **None**: Solo el valor None.

example.py

```
1 def nada(x: None):  
2     print("Llamando a nada")  
3  
4 nada(None)  
5 nada(3)
```

mypy

```
$ mypy example.py  
example.py:5: error: Argument 1 to "nada" has  
    incompatible type "int"; expected "None" [arg-type]  
Found 1 error in 1 file (checked 1 source file)
```

- **list[T]**: Lista de objetos del tipo T.

example.py

```
1 def sumar(lista: list[int]) -> int:
2     ret = 0
3     for x in lista:
4         ret += x
5     return ret
6
7 sumar([1, 2, 3])
8 sumar(['hola', 'chau'])
```

mypy

```
$ mypy example.py
example.py:8: error: List item 0 has
    incompatible type "str"; expected "int" [list-item]
example.py:8: error: List item 1 has
    incompatible type "str"; expected "int" [list-item]
Found 2 errors in 1 file (checked 1 source file)
```

- **dict[K, V]**: Diccionario de objetos de tipo K hacia objetos de tipo V.

example.py

```
1 def put(conjunto: dict[int, bool], x: int):  
2     conjunto[x] = True  
3  
4 put({1: True}, 2)  
5 put({'a': True}, 2)  
6 put({3: 'c'}, 4)
```

mypy

```
$ mypy example.py  
example.py:5: error: Dict entry 0 has  
    incompatible type "str": "bool";  
    expected "int": "bool" [dict-item]  
example.py:6: error: Dict entry 0 has  
    incompatible type "int": "str";  
    expected "int": "bool" [dict-item]  
Found 3 errors in 1 file (checked 1 source file)
```

- `tuple[T1, ..., Tn]`: *N-tupla* con elementos de los tipos `T1, ..., Tn`.

example.py

```
1 def juntar(a: str, b: int) -> tuple[str, int]:  
2     return (a, b)  
3  
4 juntar('hola', 1)  
5 juntar(2, 'chau')
```

mypy

```
$ mypy example.py  
example.py:5: error: Argument 1 to "juntar" has  
    incompatible type "int"; expected "str" [arg-type]  
example.py:5: error: Argument 2 to "juntar" has  
    incompatible type "str"; expected "int" [arg-type]  
Found 2 errors in 1 file (checked 1 source file)
```

- **Union[T1, ..., Tn]**: Admite elementos pertenecientes a alguno de los tipos T1, ..., Tn.
- **[T1 | ... | Tn]**: Equivalente a Union[T1, ..., Tn].
- **Optional[T]**: Admite elementos del tipo T y None.

Los constructores Union y Optional están definidos en el módulo typing que viene junto con MyPy.

Más info: https://mypy.readthedocs.io/en/stable/kinds_of_types.html

- **Union[T1, ..., Tn]**: Admite elementos de los tipos T1, ..., Tn.

example.py

```
1  from typing import Union
2
3  def imprimir_cond(x: Union[str, None]):
4      if x is not None:
5          print(x)
6
7  imprimir_cond('hola') # imprime "hola"
8  imprimir_cond(None)   # no imprime
9  imprimir_cond(1)      # no tipa
```

mypy

```
$ mypy example.py
example.py:9: error: Argument 1 to "imprimir_cond" has
    incompatible type "int";
    expected "str | None" [arg-type]
Found 1 error in 1 file (checked 1 source file)
```

- **[T1 | ... | Tn]**: Equivalente a `Union[T1, ..., Tn]`.

example.py

```
def imprimir_cond(x: str | None):  
    if x is not None:  
        print(x)  
  
imprimir_cond('hola') # imprime "hola"  
imprimir_cond(None)  # no imprime  
imprimir_cond(1)     # no tipa
```

mypy

```
$ mypy example.py  
example.py:9: error: Argument 1 to "imprimir_cond" has  
    incompatible type "int";  
    expected "str | None" [arg-type]  
Found 1 error in 1 file (checked 1 source file)
```

- **Optional[T]**: Admite elementos del tipo T y None.

example.py

```
1  from typing import Optional
2
3  def imprimir_cond(x: Optional[str]):
4      if x is not None:
5          print(x)
6
7  imprimir_cond('hola') # imprime "hola"
8  imprimir_cond(None)   # no imprime
9  imprimir_cond(1)      # no tipa
```

mypy

```
$ mypy example.py
example.py:9: error: Argument 1 to "imprimir_cond" has
    incompatible type "int";
    expected "str | None" [arg-type]
Found 1 error in 1 file (checked 1 source file)
```

- Mypy tiene un modo estricto.

- Mypy tiene un modo estricto.
- Habilita una serie de chequeos adicionales.

- Mypy tiene un modo estricto.
- Habilita una serie de chequeos adicionales.
- Básicamente cualquier posible error de runtime será reportado por Mypy.

- Mypy tiene un modo estricto.
- Habilita una serie de chequeos adicionales.
- Básicamente cualquier posible error de runtime será reportado por Mypy.
- Puede resultar demasiado tedioso.

- Mypy tiene un modo estricto.
- Habilita una serie de chequeos adicionales.
- Básicamente cualquier posible error de runtime será reportado por Mypy.
- Puede resultar demasiado tedioso.
- Sí, más tedioso aún. . .

example.py

```
1 def sumar(x: int, y: int):  
2     return x + y  
3  
4 sumar(1, 2) + 'hola'
```

mypy

```
$ mypy example.py  
Success: no issues found in 1 source file
```

python

```
$ python example.py  
Traceback (most recent call last):  
[...]  
    sumar(1, 2) + 'hola'  
    ~~~~~^~~~~~  
TypeError: unsupported operand type(s)  
    for +: 'int' and 'str'
```

example.py

```
1 def sumar(x: int, y: int):  
2     return x + y  
3  
4 sumar(1, 2) + 'hola'
```

mypy

```
$ mypy --strict example.py  
example.py:1: error: Function is missing a return  
    type annotation [no-untyped-def]  
Found 1 error in 1 file (checked 1 source file)
```

example.py

```
1 def sumar(x: int, y: int) -> int:  
2     return x + y  
3  
4 sumar(1, 2) + 'hola'
```

mypy

```
$ mypy example.py  
example.py:4: error: Unsupported operand types  
    for + ("int" and "str") [operator]  
Found 1 error in 1 file (checked 1 source file)
```

- Ir a la extensión Mypy Type Checker y abrir su configuración.
- Agregar el argumento `--strict` en el parámetro Mypy-type-checker: Args.

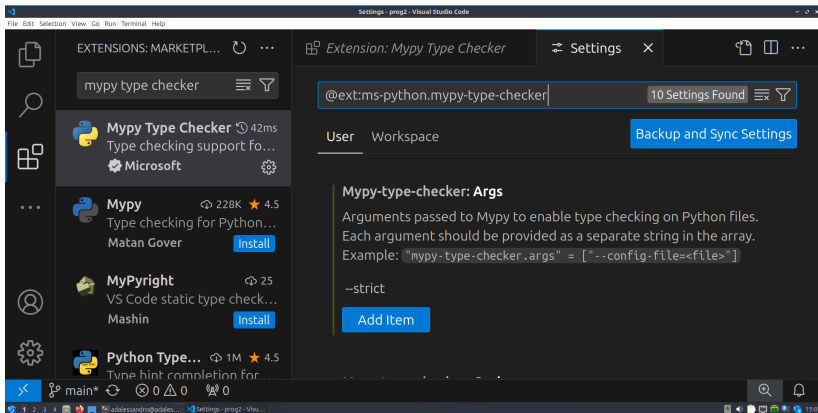


Figure 4: Mypy Type Checker - Strict mode

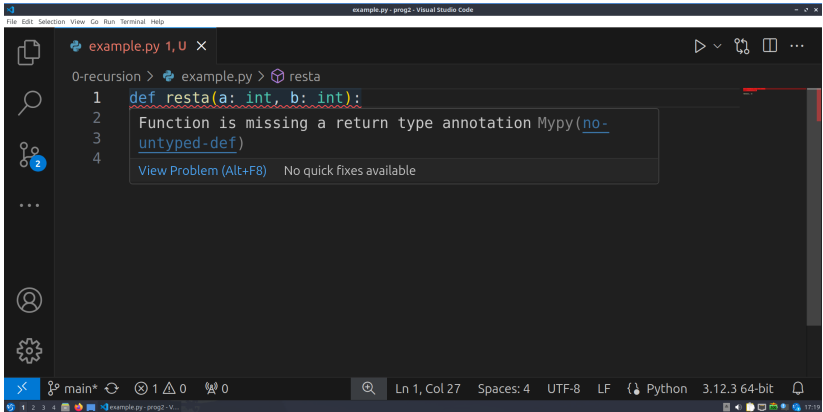


Figure 5: Mypy Type Checker - Strict mode

- Python es un lenguaje dinámicamente tipado.

- Python es un lenguaje dinámicamente tipado.
- Gracias a las *type annotations*, Mypy permite detectar posibles errores previo a la ejecución.

- Python es un lenguaje dinámicamente tipado.
- Gracias a las *type annotations*, Mypy permite detectar posibles errores previo a la ejecución.
- Se puede utilizar desde la terminal con el comando `mypy`.

- Python es un lenguaje dinámicamente tipado.
- Gracias a las *type annotations*, Mypy permite detectar posibles errores previo a la ejecución.
- Se puede utilizar desde la terminal con el comando `mypy`.
- En VSCode puede instalarse la extensión `Mypy Type Checker`.

- Python es un lenguaje dinámicamente tipado.
- Gracias a las *type annotations*, Mypy permite detectar posibles errores previo a la ejecución.
- Se puede utilizar desde la terminal con el comando `mypy`.
- En VSCode puede instalarse la extensión Mypy Type Checker.
- Podemos obtener un análisis completo con el modo `strict`.